VECTOR DATABASE-POWERED RAG PIPELINE

Group project plan • Three-person work division • Start-to-finish roadmap

Recommended project title
Building a Vector Database-Powered RAG Pipeline with LangChain and ChromaDB

Project objective
Build a functional Retrieval-Augmented Generation (RAG) application that retrieves relevant information from a knowledge base and uses an LLM to generate contextual answers. The group's central study area is the vector database.

Recommended technology stack
Python 3.9+ • LangChain • ChromaDB • all-MiniLM-L6-v2 or OpenAI embeddings • OpenAI or Ollama • Streamlit (suggested UI) • VS Code / Jupyter Notebook

Prepared as a practical group execution guide. The 14-day schedule is a suggested plan, not a deadline stated in the assignment.

1. Project overview

The project has three connected workstreams. Each person owns one workstream, but all members should understand the complete RAG pipeline and be ready to explain it.

Person 1 — Data processing & embeddings	Person 2 — Vector database & retrieval	Person 3 — RAG, LLM & evaluation
Load documents, extract text, chunk content, generate embeddings and attach metadata.	Store vectors in ChromaDB, understand indexing and implement similarity search and Top-K retrieval.	Pass retrieved context to an LLM, generate cited answers, build the UI and evaluate results.
Deliverable: Processed chunks, embeddings and metadata.	**Deliverable:** Working vector store and retriever.	**Deliverable:** Working RAG application and evaluation.

Overall system flow

Input documents
PDFs, web articles and other supported files

↓

Person 1: Ingestion
Load → extract → chunk → embed

↓

Person 2: Vector database
Store → index → similarity search → retrieve Top-K

↓

Person 3: RAG application
Format context → prompt LLM → answer with citations

↓

Final application
User asks a question and receives an answer grounded in the documents

2. Person 1 — Document processing & embeddings

Goal: Turn source documents into clean, appropriately sized text chunks with embeddings and metadata.

Topics to study

• What RAG is and why retrieval is useful.

• Document loading and text extraction for PDFs, HTML/web articles and another document type.

• Preprocessing, chunking, chunk size and overlap.

• Embeddings: what they represent and how text becomes a vector.

• Embedding models such as all-MiniLM-L6-v2 and OpenAI embedding models.

• How image, audio and video content can be represented (for example, through captions, transcripts, frame descriptions or multimodal models).

Implementation tasks

Order	Task	Suggested file
1	Load source documents	src/document_processing/load_documents.py
2	Extract and normalize text	load_documents.py
3	Split documents into chunks	src/document_processing/chunking.py
4	Generate embeddings	src/embeddings/embedding_model.py
5	Attach source/page metadata	processing output

Person 1 handoff: Provide Person 2 with a consistent set of document chunks, embeddings and metadata. Agree on the data format before implementation.

3. Person 2 — Vector database & retrieval

Goal: Store the vectors and associated information in ChromaDB, then retrieve the most relevant chunks for a query.

Topics to study

• What a vector database is; traditional databases versus vector databases.

• Records, IDs, collections, documents, embeddings and metadata.

• Persistence and insert, update, delete and query operations.

• Exact nearest-neighbour search versus approximate nearest-neighbour (ANN) search.

• HNSW: graph-based indexing and the general roles of M, ef_construction and ef_search.

• Similarity measures: cosine similarity, Euclidean distance and dot product.

• Semantic search and configurable Top-K retrieval.

• ChromaDB and alternatives such as FAISS, Milvus, Pinecone, Weaviate and pgvector.

Implementation tasks

Order	Task	Suggested file
1	Create a ChromaDB collection	src/vector_store/chroma_db.py
2	Store vectors, documents and metadata	chroma_db.py
3	Configure persistent storage	chroma_db.py
4	Implement similarity search	src/vector_store/search.py

5	Expose configurable Top-K results	search.py
6	Run retrieval experiments	notebooks/experiments.ipynb

Experiments to show: Test different Top-K values and chunking settings. Record whether relevant passages are retrieved and how long retrieval takes. If feasible, compare exact and ANN search. HNSW is useful background, but the assignment does not require implementing HNSW from scratch.

Person 2 handoff: Provide Person 3 with a function that accepts a query and returns the retrieved text chunks plus source metadata.

4. Person 3 — RAG, LLM, interface & evaluation

Goal: Use the retrieved passages as context for an LLM, present a grounded answer and evaluate system behaviour.

Topics to study

• Query embedding and retrieval flow.

• Context formatting and prompt construction.

• Zero-shot and few-shot prompting.

• Chain-of-thought and tree-of-thought concepts as named in the assignment; understand their role and limitations.

• Contextual engineering and grounding answers in retrieved evidence.

• Citations, source references and page numbers.

• Fallback behaviour when the knowledge base does not contain the answer.

• Evaluation: accuracy, faithfulness, relevance, hallucination reduction and response time.

Implementation tasks

Order	Task	Suggested file
1	Connect to Person 2's retriever	src/rag/retriever.py
2	Format retrieved context	src/rag/chain.py
3	Write and test prompts	src/rag/prompts.py
4	Connect the selected LLM	src/rag/chain.py
5	Build the simple UI	app.py
6	Run evaluation queries and record metrics	src/evaluation/

Person 3 handoff: A working question-answering interface that displays the answer and supporting document/page references, plus evaluation results.

5. Suggested 14-day execution schedule

This is a suggested schedule for a group that can work consistently. Adjust it to your course deadline and members' availability.

Day(s)	Main work	Lead	Milestone
1–2	Understand assignment, divide responsibilities, choose documents and agree on stack.	All	Plan agreed
3	Create repository, Python environment, dependencies and folder structure.	All	Environment ready
4–5	Load at least three document types, extract text, test chunking and generate embeddings.	Person 1	Ingestion works
6–7	Create ChromaDB collection, store embeddings and metadata, implement Top-K search.	Person 2	Retrieval works
8–9	Connect retriever to LLM, build prompts, citations and unsupported-question fallback.	Person 3	RAG works
10	Build simple Streamlit interface and display answers/sources.	Person 3	UI works
11–12	Test in-scope and out-of-scope questions; compare RAG with base LLM; record results.	All	Evaluation complete

Day(s)	Main work	Lead	Milestone
13	Complete README, architecture diagram, experiment notes and presentation slides.	All	Submission prepared
14	Run full demo, check citations and rehearse the 15-minute presentation.	All	Ready to present

6. Recommended project folder structure

Use one shared GitHub repository. This structure is a practical recommendation; the assignment does not prescribe exact filenames.

```
vector-database-rag/
■■■ data/
■ ■■■ pdfs/
■ ■■■ web_articles/
■ ■■■ other_documents/
■■■ src/
■ ■■■ document_processing/
■ ■ ■■■ load_documents.py
■ ■ ■■■ chunking.py
■ ■■■ embeddings/
■ ■ ■■■ embedding_model.py
■ ■■■ vector_store/
■ ■ ■■■ chroma_db.py
■ ■ ■■■ search.py
■ ■■■ rag/
■ ■ ■■■ retriever.py
■ ■ ■■■ chain.py
■ ■ ■■■ prompts.py
■ ■■■ evaluation/
■ ■■■ test_queries.py
■ ■■■ metrics.py
■■■ chroma_db/
■■■ notebooks/
■ ■■■ experiments.ipynb
■■■ app.py
■■■ requirements.txt
■■■ .env
■■■ .gitignore
■■■ README.md
```

7. Integration contract between members

Producer	Consumer	Handoff
Person 1	Person 2	Consistent chunks, embeddings and metadata; agree on data format and embedding model.
Person 2	Person 3	A query function returning relevant text chunks and source/page metadata.
Person 3	End user / group	Answer grounded in retrieved context, with citations and a fallback when evidence is missing.

Agree on these interfaces before coding in parallel. This prevents incompatible functions and duplicated work.

8. Technology stack and initial setup

Recommended language: Python 3.9 or newer, matching the assignment. Python is the main language for document processing, embeddings, ChromaDB, RAG and evaluation.

Component	Recommended choice	Why / note
Language	Python 3.9+	Specified by the assignment.
RAG framework	LangChain	Specified by the assignment.
Vector database	ChromaDB	Specified by the assignment.
Embeddings	all-MiniLM-L6-v2 or OpenAI	Both are options named in the assignment.
LLM	OpenAI or Ollama	Assignment allows OpenAI API or local alternative such as Ollama.

UI	Streamlit	Suggested simple UI; not specified as mandatory.
Development	VS Code / Jupyter	Practical development and experiment tools.

Initial Windows setup commands

```
mkdir vector-database-rag
cd vector-database-rag
python -m venv venv
venv\\Scripts\\activate
```

Create requirements.txt and install the dependencies. Package versions should be pinned after the group confirms a working combination.

```
pip install -r requirements.txt
```

Example initial requirements (review and pin compatible versions before final submission)

```
langchain
langchain-community
langchain-chroma
langchain-huggingface
sentence-transformers
chromadb
pypdf
python-dotenv
streamlit
openai
```

Keep API keys in .env. Add .env, venv/ and generated ChromaDB storage to .gitignore as appropriate. Never commit secrets to GitHub.

9. Evaluation plan

Create a small test set with questions whose answers are present in the knowledge base and questions whose answers are absent. Keep the questions and expected evidence in a shared file so all members evaluate consistently.

Test group	What to test	What to record
In-scope questions	Can the system retrieve the right passages and answer correctly?	Expected answer, retrieved sources, answer quality
Out-of-scope questions	Does the system avoid inventing an answer when the documents do not contain it?	Whether it clearly says evidence is insufficient
Base LLM vs RAG	Ask the same questions with and without retrieved context.	Correctness, grounding and unsupported claims
Retrieval settings	Change chunk size/overlap and Top-K.	Relevant passages retrieved and retrieval latency

Report actual measurements. Do not invent an accuracy percentage or claim that hallucinations were reduced unless your tests support it. Explain the test set, method and limitations.

10. 15-minute presentation plan

Time	Speaker	Content
0–3 min	Person 1	Problem, RAG overview, document loading, chunking and embeddings.
3–7 min	Person 2	Vector database, storage, metadata, indexing, similarity search and ChromaDB.
7–10 min	Person 3	Query flow, retrieval, prompts, LLM, citations and evaluation.
10–15 min	All members	Live demo and comparison of base LLM versus RAG.

Suggested live demo sequence

• Load or select the knowledge-base documents.

• Show that the documents have been indexed in ChromaDB.

• Ask a question answered by the documents.

• Show the retrieved chunks and their source/page metadata.

• Show the generated answer with citations.

• Ask a question outside the knowledge base and demonstrate the fallback.

• Briefly show one evaluation result comparing base LLM and RAG.

11. Final submission checklist

- ■ At least three document types are loaded and processed.
- ■ Chunk size and overlap experiments are documented.
- ■ Embeddings are generated with a selected model.
- ■ ChromaDB stores vectors, documents and metadata.
- ■ Similarity search and configurable Top-K work.
- ■ The RAG pipeline passes retrieved context to an LLM.
- ■ Answers include citations/source references.
- ■ Unsupported questions trigger a sensible fallback.
- ■ Evaluation includes in-scope and out-of-scope questions.
- ■ RAG is compared with a base LLM on the same questions.
- ■ Response time and actual results are recorded.
- ■ Python scripts or a documented Jupyter notebook are included.
- ■ Architecture diagram and design decisions are explained.
- ■ All members can explain the complete pipeline.

12. Important notes and assumptions

Requirements taken from the professor's assignment

The assignment calls for Python 3.9+, LangChain loaders, at least three document types, chunking experiments, embeddings, ChromaDB with metadata, similarity/semantic search with configurable K, a simple UI, cited answers, prompting experiments, inside/outside knowledge-base testing, base-LLM comparison and a 15-minute presentation with a 5-minute demo.

Recommendations added to make the group plan practical

The three-person division, Streamlit, exact folder names, GitHub branch names and 14-day schedule are recommendations. HNSW is a useful indexing concept to study, but the assignment does not explicitly require implementing it. Select one common dataset and one embedding model for all members so the components remain compatible.

Suggested knowledge-base theme

A useful option is to build the knowledge base around vector databases and RAG itself: documents covering embeddings, ChromaDB, indexing, HNSW, similarity metrics and retrieval. This is a recommendation, not a topic specified by the assignment. Confirm the dataset/theme with your professor or group before collecting documents.

End of group project plan